



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ (ИУ7)
НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.04 Программная инженерия

О т ч е т
по практикуму № 1

Название: **Разработка и отладка программ в вычислительном комплексе**

Тераграф

Дисциплина: **Архитектура ЭВМ**

Студент гр. ИУ7-52Б

(Подпись, дата)

М. А. Жаринов

(И.О. Фамилия)

Преподаватель

(Подпись, дата)

М.А. Гейне

(И.О. Фамилия)

2025 год

Цель работы: освоение принципов работы вычислительного комплекса Тераграф и получение практических навыков решения задач обработки множеств на основе гетерогенной вычислительной структуры. В ходе практикума необходимо ознакомиться с типовой структурой двух взаимодействующих программ: хост-подсистемы и программного ядра `sw_kernel`. Для выполнения практикума предоставляется доступ к облачной платформе `devlab.bmstu.ru` с установленными ускорительными картами микропроцессора Леонард Эйлер и настроенными средствами сборки проектов.

Выполнение практикума

Необходимо было разработать программу для хост-подсистемы и обработчики программного ядра, выполняющие следующие действия по моему варианту (34):

Сформировать в хост-подсистеме и передать в SPE две коллекции. Описание коллекций:

- `students`: `student_id`, `name`, `year_of_study`.
- `course_materials`: `material_id`, `course_id`, `year_available`, `title`.

Задание:

Получить в хост-подсистеме и выдать на экран названия учебных материалов, доступных для студентов второго курса, таких как Елена Гордеева (передается в запросе из хост-подсистемы)?

Эквивалентный запрос на языке AQL:

```
FOR student IN students
  FILTER student.name == "Елена Гордеева" AND
student.year_of_study == 2
  FOR material IN course_materials
    FILTER material.year_available <=
student.year_of_study
  RETURN material.title
```

Код для выполнения практикума базировался на примере №4, изменялись только файлы `host_main.cpp`, `sw_kernel_main.cpp`, `common_struct.h`, `gpc_handlers.h`.

Исходный код host_main.cpp:

```
#include <iostream>
#include <string>
#include <ctime>
#include <cstring>
#include <vector>
#include <map>
#include "host_main.h"

using namespace std;

// Функция для ввода строки с дополнением нулями
void input_padded_name(const char* prompt, char* buffer, int buffer_size)
{
    printf("%s: ", prompt);
    char temp[STR_LENGTH];
    fgets(temp, sizeof(temp), stdin);
    temp[strcspn(temp, "\n")] = '\0';
    strncpy(buffer, temp, buffer_size - 1);
    buffer[buffer_size - 1] = '\0';
    for (int i = strlen(buffer); i < buffer_size; i++) {
        buffer[i] = '\0';
    }
}

// Заполнение строки нулевыми символами
void fill_name_full(char dest[STR_LENGTH], const char* first_name, const
char* last_name) {
    memset(dest, 0, STR_LENGTH);
    snprintf(dest, STR_LENGTH, "%s %s", first_name, last_name);
}

// Заполнение строки нулевыми символами
void fill_title_full(char dest[STR_LENGTH], const char* name, uint64_t
year_available) {
    memset(dest, 0, STR_LENGTH);
    snprintf(dest, STR_LENGTH, "%s для %lu купца", name, year_available);
}

int main(int argc, char** argv) {
    gpc *gpc64_inst = new gpc();
    printf("Open gpc on %s\n", gpc64_inst->gpc_dev_path);
    if (gpc64_inst->load_swk(argv[1])) {
        printf("Rawbinary %s file error\n", argv[1]);
        delete gpc64_inst;
        return -1;
    }
    printf("Rawbinary loaded from %s\n", argv[1]);

    MessageProcessor<students_Key, students_Value> students(gpc64_inst);
    MessageProcessor<course_materials_Key, course_materials_Value>
course_materials(gpc64_inst);
    char str_buf[STR_LENGTH];

    // srand(time(NULL));
```

```

const char* first_names[] = {
    "Александр", "Иван", "Сергей", "Дмитрий", "Андрей", "Михаил",
    "Алексей", "Артем", "Кирилл", "Максим", "Егор", "Никита",
    "Павел", "Роман", "Владимир", "Олег", "Тимофей", "Глеб",
    "Даниил", "Константин", "Евгений", "Илья", "Антон", "Матвей",
    "Степан", "Ярослав", "Борис", "Юрий", "Виктор", "Василий",
    "Валерий", "Георгий", "Григорий", "Денис", "Захар", "Игорь",
    "Лев", "Леонид", "Николай", "Петр", "Руслан", "Семен",
    "Станислав", "Федор", "Эдуард", "Юлиан", "Ян", "Арсений",
    "Вадим", "Виталий", "Вячеслав", "Геннадий", "Давид", "Елисей",
    "Игнат", "Клим", "Лука", "Марк", "Назар", "Оскар",
    "Платон", "Проход", "Радислав", "Родион", "Савелий", "Тарас",
    "Тимур", "Филипп", "Эрик", "Альберт", "Аркадий", "Богдан",
    "Влад", "Гарри", "Демид", "Емельян", "Ждан", "Зиновий",
    "Ираклий", "Камиль", "Людвиг", "Марсель", "Нестор", "Орест",
    "Панкрат", "Ростислав", "Святослав", "Тихон", "Ульян", "Фаддей",
    "Харитон", "Цезарь", "Чеслав", "Эмиль", "Яков",
    "Адам", "Бронислав", "Варлам", "Гавриил", "Добрыня", "Ефрем",
    "Елена"
};

const char* last_names[] = {
    "Иванов", "Петров", "Сидоров", "Смирнов", "Кузнецов", "Попов",
    "Васильев", "Павлов", "Семенов", "Голубев", "Виноградов", "Богданов",
    "Воробьев", "Федоров", "Михайлов", "Белов", "Тарасов", "Беляев",
    "Гришин", "Титов", "Комаров", "Орлов", "Соловьев", "Лебедев",
    "Козлов", "Новиков", "Морозов", "Волков", "Алексеев", "Егоров",
    "Соколов", "Борисов", "Яковлев", "Герасимов", "Фомин", "Давыдов",
    "Жуков", "Наумов", "Ширяев", "Бирюков", "Копылов", "Крылов",
    "Терентьев", "Молчанов", "Владимиров", "Артемьев", "Гурьев", "Зимин",
    "Кудрявцев", "Буров", "Зуев", "Трофимов", "Сафонов", "Щербаков",
    "Дорофеев", "Нестеров", "Харитонов", "Агафонов", "Муравьев",
    "Ларионов",
    "Федосеев", "Зиновьев", "Капустин", "Сысоев", "Хохлов", "Жданов",
    "Галкин", "Пономарев", "Рябов", "Суханов", "Фролов", "Никонов",
    "Селиверстов", "Чернов", "Шестаков", "Дементьев", "Костин",
    "Софронов",
    "Третьяков", "Рогов", "Лыткин", "Токарев", "Устинов", "Черкасов",
    "Шубин", "Щукин", "Юдин", "Якушев", "Абрамов", "Белоусов",
    "Власов", "Горбунов", "Дроздов", "Ермаков", "Журавлев", "Зайцев",
    "Гордеева"
};

const char *course_titles[] = {
    "Учебник по матанализу",
    "Пособие по электротехнике",
    "Сборник задач по физике",
    "Методические указания по химии",
    "Курс лекций по философии",
    "Учебник по истории России",
    "Введение в программирование на C",
    "Основы микроэкономики",
    "Справочник по высшей математике",
    "Лабораторный практикум по биологии",
    "Учебник по английскому языку",

```

```

        "Задачник по теоретической механике",
        "Пособие по начертательной геометрии",
        "Конспект лекций по психологии",
        "Атлас по географии"
    };

    int name_cnt = sizeof(first_names) / sizeof(first_names[0]);
    int lastname_cnt = sizeof(last_names) / sizeof(last_names[0]);
    uint64_t course_titles_cnt = sizeof(course_titles) /
sizeof(course_titles[0]);

    students_Key sk;
    students_Value sv;
    course_materials_Key cmk;
    course_materials_Value cmv;

    for (int i = 0; i < name_cnt; i++) {
        for (int j = 0; j < lastname_cnt; j++) {
            uint64_t student_id = i * lastname_cnt + j + 1;

            fill_name_full(str_buf, first_names[i], last_names[j]);
            reverse_string(sk.name, str_buf, sizeof(sk.name));
            sk.student_id = student_id;
            sv.year_of_study = (rand() % MAX_COURSE) + 1;

            gpc64_inst->start(__event__(insert_student));
            students.send_sync(sk, sk.size(), sv, sv.size());
            if (i == 0 && j == 1)
                printf("Запись идет успешно\n");
        }
    }

    uint64_t material_id = 0;
    for (uint64_t course_id = 1; course_id < course_titles_cnt + 1;
course_id++) {
        uint64_t n_courses = (rand() % MAX_COURSE) + 1;
        uint64_t start_course = rand() % (MAX_COURSE - n_courses + 1);
        for (uint64_t year_available = start_course; year_available <
start_course + n_courses; year_available++)
        {
            material_id++;
            fill_title_full(str_buf, course_titles[course_id - 1],
year_available);
            reverse_string(cmv.title, str_buf, sizeof(cmv.title));

            cmv.course_id = course_id;
            cmk.material_id = material_id;
            cmk.year_available = year_available;

            gpc64_inst->start(__event__(insert_course_material));
            course_materials.send_sync(cmk, cmk.size(), cmv, cmv.size());
        }
    }

    input_padded_name("Введите ФИ (например, Елена Гордеева)", sk.name,
STR_LENGTH);

```

```

        reverse_string(sk.name, sizeof(sk.name));

        gpc64_inst->start(__event__(ask_materials));
        students.send_sync(sk, sk.size(), sv, sv.size());

        bool student_found = gpc64_inst->mq_receive();
        if (!student_found)
        {
            printf("Введенный студент не найден!\n");
        }
        else
        {
            uint64_t student_course = gpc64_inst->mq_receive();
            printf("Курс у студента: %lu\n", student_course);
            course_materials.receive_sync(cm_k, cm_v);
            if (cm_k.material_id == -1ull)
            {
                printf("Материалы не найдены!\n");
            }
            else
            {
                printf("Доступные материалы:\n");
                do
                {
                    reverse_string(str_buf, cm_v.title, STR_LENGTH);
                    printf("\t%s\n", str_buf);
                    course_materials.receive_sync(cm_k, cm_v);
                } while (cm_k.material_id != -1ull);
            }
        }

        delete gpc64_inst;
        return 0;
    }
}

```

Исходный код sw_kernel_main.cpp:

```
#include <stdlib.h>
#include <string.h>
#include <ctime>
#include "lnh64.hxx"
#include "gpc_io_swk.h"
#include "gpc_handlers.h"
#include "common_struct.h"
#include "chains.hxx"

extern lnh lnh_core;
volatile unsigned int event_source;

int main(void) {
    ///////////////////////////////////////////////////
    //                               Main Event Loop
    ///////////////////////////////////////////////////
    //Leonhard driver structure should be initialised
    lnh_init();
    for (;;) {
        //Wait for event
        event_source = wait_event();
        switch(event_source)
        {
            case __event__(insert_student) : insert_student(); break;
            case __event__(insert_course_material) :
insert_course_material(); break;
            case __event__(ask_materials) : ask_materials(); break;
        }
        set_gpc_state(READY);
    }
}

//-----
//      Вставка ключа и значения в структуру
//-----

MessageProcessor<students_Key, students_Value> students(1);
MessageProcessor<students_Key, students_Value> student_requested(1);
MessageProcessor<course_materials_Key, course_materials_Value>
course_materials(2);

void insert_student() {
    students.receive();
    students.insert();
}

void insert_course_material() {
    course_materials.receive();
    course_materials.insert();
}

void ask_materials () {
```

```

    student_requested.receive();

    students.key(student_requested.key());
    students.key().student_id = student_requested.key().student_id = -1ull;

    if (students.nsm(FIELD_U64_ROFFSET(students_Key, student_id)) &&
    streq(student_requested.key().name, students.key().name,
    sizeof(students.key().name)))
    {
        mq_send(1ull);
        uint64_t stud_year = students.value().year_of_study;
        mq_send(stud_year);
        for (uint64_t year = 1; year <= stud_year; year++)
        {
            course_materials.key().year_available = year;
            course_materials.key().material_id = -1ull;
            while
(course_materials.nsm(FIELD_U64_ROFFSET(course_materials_Key, material_id))
&& course_materials.key().year_available == stud_year)
            {
                course_materials.send();
            }
            course_materials.key().material_id = -1ull;
            course_materials.send();
        } else
        {
            mq_send(0ull);
        }
    }
}

```


Исходный код common_struct.h:

```
#ifndef COMMON_STRUCT
#define COMMON_STRUCT

#include "common_defs.h"

////////////////////////////////////
// Описание формата ключа и значения
////////////////////////////////////

// Сформировать в хост-подсистеме и передать в SPE две коллекции. Описание
коллекций:
// students: student_id, name, year_of_study.

#define MAX_COURSE 11
#define STR_LENGTH 128

RALIGNED_STRUCT (students_Key,
    uint64_t    student_id = 0;
    char        name[STR_LENGTH] = { 0 };
)

LALIGNED_STRUCT (students_Value,
    uint64_t    year_of_study = 0;
)

// course_materials: material_id, course_id, year_available, title.

RALIGNED_STRUCT (course_materials_Key,
    uint64_t    material_id = 0;
    uint64_t    year_available = 0;
)

LALIGNED_STRUCT (course_materials_Value,
    uint64_t    course_id = 0;
    char        title[STR_LENGTH] = { 0 };
)

#endif
```

Исходный код gpc_handlers.h:

```
#pragma once

#include "common_defs.h"

// Декларация обработчиков
DECLARE_EVENT_HANDLER(insert_student);
DECLARE_EVENT_HANDLER(insert_course_material);
DECLARE_EVENT_HANDLER(ask_materials);
```

Результат работы программы:

```
iu7034@dl580:~/lab1$ host/host_main sw-kernel/sw_kernel.rawbinary
Open gpc on /dev/gpc1
Rawbinary loaded from sw-kernel/sw_kernel.rawbinary
Запись идет успешно
Введите ФИ (например, Елена Гордеева): Иван Морозов
Курс у студента: 4
Доступные материалы:
    Атлас по географии для 1 курса
    Конспект лекций по психологии для 1 курса
    Лабораторный практикум по биологии для 1 курса
    Справочник по высшей математике для 1 курса
    Курс лекций по философии для 1 курса
    Сборник задач по физике для 1 курса
    Атлас по географии для 2 курса
    Конспект лекций по психологии для 2 курса
    Лабораторный практикум по биологии для 2 курса
    Справочник по высшей математике для 2 курса
    Основы микроэкономики для 2 курса
    Курс лекций по философии для 2 курса
    Атлас по географии для 3 курса
    Конспект лекций по психологии для 3 курса
    Пособие по начертательной геометрии для 3 курса
    Лабораторный практикум по биологии для 3 курса
    Справочник по высшей математике для 3 курса
    Основы микроэкономики для 3 курса
    Курс лекций по философии для 3 курса
    Атлас по географии для 4 курса
    Конспект лекций по психологии для 4 курса
    Пособие по начертательной геометрии для 4 курса
    Учебник по английскому языку для 4 курса
    Лабораторный практикум по биологии для 4 курса
    Справочник по высшей математике для 4 курса
    Основы микроэкономики для 4 курса
    Методические указания по химии для 4 курса
    Учебник по матанализу для 4 курса
```

Вывод

В процессе выполнения практикума я освоил принципы работы вычислительного комплекса Тераграф и получил практические навыки решения задач обработки множеств на основе гетерогенной вычислительной структуры. Я ознакомился с типовой структурой двух взаимодействующих программ: хост-подсистемы и программного ядра `sw_kernel`.